Gate 3 — Dashboard Architecture Review

Status: Proposal (2026-04-21) **Branch:** `docs/gate-3-arch-review` **Scope:** Resolve the five open questions in `docs/planning/registry-redesign/gate-3-dashboard.md` and recommend a data-access shape per panel. **Constraints:** D3-1..D3-6 locked. Demo hard deadline ABQ BioPark June 1–5, 2026. No schema work blocks Gate 3 (Gate 2 close-out confirmed).

This review is recommendation-first. Each section leads with the call, then reasoning, then alternatives considered. Anything tagged **Decision needed** requires Alex's signoff before the first panel PR lands.

Schema facts the review is built on

Confirmed by reading `backend/species/models.py`, `backend/populations/models.py`, `backend/fieldwork/models.py`, `backend/accounts/models.py`, and `backend/species/views_dashboard.py`:

- `Species` — has `iucn_status` (mirror), `family`, `genus_fk`, `endemic_status`, `updated_at`, `created_at`. Mirror-policy protects `iucn_status` (see CLAUDE.md). `ex_situ_populations` reverse accessor exists.
- `ExSituPopulation` — has `species` FK, `institution` FK, `breeding_status` choices (`breeding` / `non-breeding` / `unknown`), **`studbook_managed: BooleanField`** (already there, previously unnoticed in the spec), `last_census_date`, `count_total`, `created_at`. **No `updated_at` field.** Unique on `(species, institution)`.
- `HoldingRecord` — census-level child of `ExSituPopulation`. Has `date`, `created_at`, reporter FK. **No `updated_at`.**
- `ConservationAssessment` — has `category`, `review_status`, `source`, `created_at`. **No `updated_at`.** Review-status transitions are implicit (no audit trail on the row itself).
- `FieldProgram` — M2M `focal_species`, `status`, no timestamps.
- `AuditLog` — **already exists** in `backend/accounts/models.py`: `(user, action ∈ {create,update,delete}, model_name, object_id, timestamp, changes: JSON, ip_address)`. Indexed by default ordering `-timestamp`. Whether it is actually being *written to* across the codebase is the open question (see Q4).
- Existing `DashboardView` lives at `species/views_dashboard.py` and already computes threatened / captive overlap — Gate 3 panels can sit next to it as sibling APIViews.

These facts change two spec assumptions:

1. `ExSituPopulation` has **no `updated_at`** — Panel 4 cannot union it in without a migration.
2. `studbook_managed` **already exists** — Q3 is a narrower question than the spec framed.

Open question resolutions

Q1 — Coverage gap scope: include DD?

Recommendation: Exclude DD from Panel 1 for Gate 3. Do not build a separate "needs assessment" bucket now. Leave DD visibility to the Red List snapshot (Panel 5) and the directory filter rail.

Reasoning.

- Panel 1's job is "here's a species that needs a breeding program." DD species are "here's a species we don't know enough about to recommend anything." Conflating them weakens the coordinator

prompt — a DD row tells a coordinator to fund a survey, not to found a population.
- A second bucket doubles the Panel 1 empty-state surface area, doubles the copy pass, and doubles the test matrix for zero qualitative lift before ABQ.
- Cutting DD in keeps the query trivial: `iucn_status__in=['CR','EN','VU']` stays a constant tuple.
- Post-ABQ, if SHOAL or a coordinator asks for "what do we not know?", add a sibling Panel 1b: "Assessment gap" — DD + `iucn_status IS NULL` species with no recent `ConservationAssessment`. That panel is its own story and should not muddy the coverage prompt.

Alternatives considered.

- *Fold DD into Panel 1.* Rejected — conflates two different coordinator asks into one sort order (what's the IUCN severity of DD?).
- *Add a separate "needs assessment" bucket in Panel 1.* Deferred — good idea, wrong gate.

Q2 — Single-institution risk sort order

Recommendation: Sort by `institution_count` ascending, then by IUCN severity (CR → EN → VU → NT → LC → DD) as tiebreaker, then alphabetical by `scientific_name`.

Reasoning.

- Panel 2's headline signal is *fragility*, not *imperilment* — that's what separates it from Panel 1 and from Panel 5. A single-institution LC species is more fragile than a two-institution CR species in ex-situ terms, even though the CR species matters more in absolute conservation terms. The panel's job is to flag the fragility; severity is secondary context that rides along.
- IUCN-severity-first would duplicate Panel 1's ordering and make Panel 2 feel like "Panel 1 but with one institution added" rather than a distinct surface.
- Severity as tiebreaker still surfaces "CR species held at one zoo" at the top of the 1-institution block, which is the most actionable row in the panel.

Alternatives considered.

- *IUCN severity first.* Rejected — duplicates Panel 1 semantics and buries the fragility signal.
- *Weighted composite score (e.g. `severity_rank × 1/institution_count`).* Rejected — opaque to coordinators, hard to explain in the ABQ demo, premature optimization. Revisit if a coordinator asks for it.

Q3 — Studbook status definition

Recommendation: Use the **existing `ExSituPopulation.studbook_managed` boolean** as the authoritative signal for "coordinated breeding." Do **not** add a `coordinated_program` field or an FK to `FieldProgram`. Panel 3's three buckets become:

- **Studbook-managed breeding** — species with ≥ 1 `ExSituPopulation` where `studbook_managed=True` AND `breeding_status='breeding'`.
- **Non-coordinated holdings** — species with ≥ 1 `ExSituPopulation` that does not meet the above (any holding without a studbook, or a studbook without breeding status).
- **No captive population** — same cohort as Panel 1's base set (filtered to threatened in the panel query, or shown across all species — see decision below).

Reasoning.

- `studbook_managed` already ships and is already surfaced in admin. Re-using it costs zero migrations and no data backfill beyond whatever population curation is already planned.

- `breeding_status='breeding'` alone is too loose — a hobbyist breeder who ticks "breeding" in good faith shows up identical to an AZA SSP. The whole point of Panel 3 is *separating* those.
- `FieldProgram` is in-situ work (surveys, habitat, community management). Wiring it into an ex-situ studbook bucket would muddy the semantics and force coordinators to use field-program records as a coordination proxy they don't actually mean.

Decision needed: Does Panel 3's "no captive population" bucket count *threatened* species only (matching Panel 1's scope) or *all* endemic species? Recommend: threatened-only, so the three buckets sum to the CR/EN/VU denominator and the panel tells a coherent coverage story.

Alternatives considered.

- *`breeding_status='breeding'` alone.* Rejected — can't distinguish ad-hoc from coordinated.
- *New `coordinated_program` boolean on `ExSituPopulation`.* Rejected as redundant with `studbook_managed`.
- *FK to `FieldProgram`.* Rejected — wrong domain (in-situ vs ex-situ).

Q4 — "Recently updated" window + source

Recommendation: 30-day window. Union of three `updated_at` / `created_at` sources, NOT `AuditLog`, for Gate 3. Add one migration to give `ExSituPopulation` and `ConservationAssessment` a `updated_at` field so the union is uniform. File the AuditLog path as a Gate-4 follow-up.

Sources to union (after the small migration):

1. `Species.updated_at` where delta includes conservation-relevant fields (see below) — scoped by filtering to rows whose most recent `ConservationAssessment` or `iucn_status` changed in window. Pragmatic shortcut: use `Species.updated_at` directly; accept that cosmetic edits surface. Worth the simplicity pre-ABQ.
2. `ExSituPopulation.updated_at` (new) — covers census updates, breeding-status flips, studbook toggles.
3. `ConservationAssessment.created_at` — new assessments are inherently conservation-relevant; an assessment row never meaningfully "updates," it supersedes.

Panel 4 row schema: `{timestamp, entity_type ∈ {species, population, assessment}, entity_label, entity_url, summary_phrase}`. `summary_phrase` is derived at query time ("Census updated", "New IUCN assessment", "Population added") — no stored change log, no actor field (skip the tier-gated actor for Gate 3).

Reasoning — why not AuditLog.

- `AuditLog` exists as a model but its write surface across the codebase is unverified. Relying on it for a demo-blocking panel is a bet against coverage we haven't audited. A post-Gate-3 audit can confirm which mutations actually emit AuditLog rows; if the coverage is spotty, we need backfill before the panel can be trusted.
- AuditLog's `changes: JSON` is a diff blob, not a human summary. Rendering it as a coordinator-readable row ("Census updated from 12 to 15") requires a presentation layer we don't have yet.
- Ordering: `AuditLog.timestamp` with `-timestamp` is already the default. Once we trust the write coverage, swapping Panel 4's data source is a one-endpoint change — the frontend contract stays identical.

Reasoning — why 30 days.

- Coordinator QA is a weekly-to-monthly activity. 30 days matches the "what's new since I last looked" mental model without the list going empty in slow weeks (at current ~79 species, a 30-day

window should have 5–15 rows in healthy operation).
- 60 and 90 are both defensible; 30 fails louder (empty state fires earlier) which is a better signal during curation.

Conservation-relevant events for Gate 3 summary-phrase mapping:

- Species: `updated_at` delta where change touched `iucn_status`, `population_trend`, `cares_status`, `shoal_priority`, `endemic_status`, `distribution_narrative`. Implementation shortcut: don't diff — just show "Species record updated" with timestamp; accept noise. Fix via AuditLog in Gate 4.
- ExSituPopulation: new row, breeding-status change, studbook-toggle, census update. Gate 3: show "Population updated at {institution}" generically.
- ConservationAssessment: new row only. Phrase: "New {source} assessment: {category}".

Migration required. Add `updated_at = models.DateTimeField(auto_now=True)` to `ExSituPopulation` and `ConservationAssessment`. `auto_now` back-populates as `now()` on first save post-migration; acceptable for a recent-activity panel (first 30 days post-migration will light up with a synthetic wave, then settle).

Decision needed: Approve the one-field migration on two models, or defer Panel 4 to post-ABQ.

Alternatives considered.

- *Use `AuditLog` directly.* Deferred — audit the write coverage first.
- *Use only `Species.updated_at`*. Would ship without a migration but misses the population-census signal, which is the highest-value row type for coordinators.
- *90-day window.* Defensible; 30 chosen for louder failure.

Q5 — KBA hook stubbing on Panel 1

Recommendation: Do not stub `kba_coverage` on the Panel 1 response for Gate 3. Frontend leaves room for the field (column-shape TODO comment, nothing more).

Reasoning.

- The KBA overlay (`docs/planning/ideation/kba-overlay.md`) is post-ABQ and has an open question about whether `ProtectedArea` (WDPA, already in the codebase) covers 80% of the use case. Stubbing `kba_coverage: bool | null` now commits to a name, a semantics, and a nullable contract before either of those questions is answered.
- The field would be `null` for every row at ABQ (no KBA data imported), which makes it dead weight on the wire and a distraction in the demo if anyone inspects the JSON.
- Post-ABQ wiring cost is genuinely cheap: a single `.annotate(kba_coverage=Exists(...))` on the Panel 1 queryset and one extra column on the frontend table. The "wire now = cheap later" argument undersells how cheap "wire later" also is when the query is already a simple queryset.
- YAGNI wins when the feature is one gate away AND the semantics aren't locked.

What to do instead. One-line comment in Panel 1's serializer: `# TODO(post-ABQ): annotate kba_coverage once KBA overlay lands — see docs/planning/ideation/kba-overlay.md`. That's the whole stub.

Alternatives considered.

- *Stub `kba_coverage: null` everywhere.* Rejected — commits to a field name and null-semantics prematurely; risks the post-ABQ design choosing a different shape (e.g. `kba_ids: [int]` instead of a boolean).

- *Stub as* `kba_ids: []`. Same objection.

Data-access shape per panel

D3-5 locks one endpoint per panel. Below is the recommended query shape, index needs, and N+1 risks for each. All panels are read-only, AllowAny (Tier 1), and should be cached with the same 5-minute TTL pattern already used by `DashboardView` (`cache.get` / `cache.set` with per-panel keys: `api:dashboard:panel-<n>:v1`).

Panel 1 — `/api/v1/dashboard/coverage-gap/`

```
Species.objects
    .filter(iucn_status__in=['CR', 'EN', 'VU'])
    .annotate(captive_count=Count('ex_situ_populations'))
    .filter(captive_count=0)
    .select_related('genus_fk')
    .order_by(
        Case(When(iucn_status='CR', then=0),
             When(iucn_status='EN', then=1),
             When(iucn_status='VU', then=2)),
        'scientific_name',
    )
    .values('id', 'scientific_name', 'family', 'iucn_status', 'endemic_status')
```

- **Index need:** existing `iucn_status` is not indexed but cardinality is ~9 values across ~79 rows — skip. The `ex_situ_populations__species` FK is already indexed (FK default). No new index.
- **N+1 risk:** none if we stick to `.values()` or a flat serializer. Adding `primary_basin` (spec mentions it but there is no such field) requires a subquery or a denormalized column — **drop `primary_basin` from the Panel 1 response for Gate 3** and note in the serializer that it's pending a product decision on what "primary basin" means when a species has multiple localities.
- **Response shape:** flat list of ~10–40 rows. Stable across a cache window.

Panel 2 — `/api/v1/dashboard/single-institution-risk/`

```
Species.objects
    .annotate(
        institution_count=Count('ex_situ_populations__institution', distinct=True),
        total_individuals=Sum('ex_situ_populations__count_total'),
    )
    .filter(institution_count__gt=0, institution_count__lte=2)
    .order_by(
        'institution_count',            # fragility first
        Case(When(iucn_status='CR', then=0),
             When(iucn_status='EN', then=1),
             When(iucn_status='VU', then=2),
             When(iucn_status='NT', then=3),
             default=4),
        'scientific_name',
    )
```

```
    .values('id', 'scientific_name', 'iucn_status', 'institution_count',
'total_individuals')
```

- **Index need:** `ExSituPopulation(species, institution)` is already unique-constrained, which gives us the composite index. No new index.
- **N+1 risk:** none — both aggregates are single-pass annotations.
- **Note:** `Sum('...count_total')` can be `None` across nullable rows; coerce to 0 in the serializer.

Panel 3 — `/api/v1/dashboard/studbook-status/`

```
# One query, three COUNT(DISTINCT species_id) branches via conditional aggregation.
from django.db.models import Q, Count

threatened = Species.objects.filter(iucn_status__in=['CR', 'EN', 'VU'])

studbook_breeding = threatened.filter(
    ex_situ_populations__studbook_managed=True,
    ex_situ_populations__breeding_status='breeding',
).distinct().count()

any_holding =
threatened.filter(ex_situ_populations__isnull=False).distinct().count()

non_coordinated = any_holding - studbook_breeding
no_population = threatened.count() - any_holding
```

- **Index need:** none.
- **N+1 risk:** none — three `count()` calls, each a single aggregate query. Four queries total for the panel (including the base threatened count), acceptable.
- **Shape:** returns `{studbook_breeding: int, non_coordinated: int, no_population: int, total_threatened: int}` plus three click-through URLs to pre-filtered directory pages.

Panel 4 — `/api/v1/dashboard/recently-updated/`

```
since = timezone.now() - timedelta(days=30)

species_rows = Species.objects.filter(updated_at__gte=since).values(
    'id', 'scientific_name', 'updated_at',
)
population_rows =
ExSituPopulation.objects.filter(updated_at__gte=since).select_related(
    'species', 'institution',
).values(
    'id', 'species__scientific_name', 'institution__name', 'updated_at',
)
assessment_rows =
ConservationAssessment.objects.filter(created_at__gte=since).select_related(
    'species',
).values(
    'id', 'species__scientific_name', 'category', 'created_at',
```

```
)

# Union in Python, sort by timestamp desc, cap at 20.
```

- **Index need:** add `db_index=True` on the two new `updated_at` fields (`ExSituPopulation` , `ConservationAssessment`) in the same migration that adds them. Cheap; supports the range filter.
- **N+1 risk:** none — each queryset selects the FK label inline via `__scientific_name` / `__name` . Three queries total, bounded by the 30-day window size.
- **Cap:** 20 rows post-sort. The panel isn't a changelog; it's a curator nudge.

Panel 5 — Red List snapshot (demoted)

No change. Existing `/api/v1/species/counts/` or the aggregation inside `DashboardView.iucn_counts` continues to power the bar chart. Frontend moves the component to the bottom of the page.

Caching + revalidation

- Per-panel cache keys, 5-minute TTL, mirroring the existing `DashboardView` pattern. Each panel's 5-min refresh is independent — D3-5's "per-panel revalidation" promise holds.
- Invalidation is implicit (TTL only) for Gate 3. Post-ABQ: add a `post_save` / `post_delete` signal on `ExSituPopulation` and `ConservationAssessment` that busts relevant keys, so a curator's edit appears within seconds rather than within minutes. Not a blocker.

Risks & open items for the PM pass

1. **`ExSituPopulation.updated_at` migration** — one-liner per model, but any migration on a pre-June codebase deserves an explicit signoff. Recommend landing this migration in the Panel 4 PR, not ahead of it.
2. **`primary_basin` in Panel 1** — spec references a field that doesn't exist. Drop for Gate 3 or wire to a computed "most-localities basin" subquery. Recommend drop; document in the Panel 1 PR description.
3. **Panel 3 denominator** — does "no captive population" mean threatened-only or all species? Recommend threatened-only; needs PM signoff.
4. **AuditLog write coverage** — independent of Gate 3, worth an audit ticket. If it turns out AuditLog is already covering all mutation paths, Panel 4 can switch to it in a one-endpoint PR post-ABQ.
5. **Cache-bust on content change** — explicit out-of-scope for Gate 3; filed for post-ABQ.
6. **`studbook_managed` data quality** — the field exists but we haven't verified current seed / production values. If it's all `False` everywhere, Panel 3's "studbook-managed breeding" bucket will be empty at ABQ. Recommend a seed-data pass as part of Panel 3's PR: mark known-coordinated populations accordingly before demo day.

Summary of recommendations

#	Question	Recommendation
1	Coverage gap scope	CR/EN/VU only; no DD; no "needs assessment" bucket in Gate 3.

2	Panel 2 sort	`institution_count` asc → IUCN severity → alphabetical.
3	Studbook definition	Use existing `studbook_managed=True AND breeding_status='breeding'`. No new field. No FieldProgram FK.
4	Recently updated	30-day window; union of `Species`/`ExSituPopulation`/`ConservationAssessment` timestamps; one migration to add `updated_at` to the latter two. AuditLog deferred.
5	KBA stub	Don't stub. One-line TODO comment only.

Decision owners: Alex to signoff on the Panel 4 migration (Q4) and the Panel 3 denominator scope (Q3 follow-up) before the first panel PR lands.